

C++ Bitset and its Application

Last Updated : 21 Aug, 2025



Bitset is a container that represents a fixed-size sequence of bits. A **bitset** allows you to manipulate individual bits efficiently, making it useful in problems related to **bitwise operations**, such as **checking flags, implementing binary representations**.

Bitset is defined as the **std::bitset** class template inside the **<bitset>** header file.

Syntax of Bitset

```
bitset<n> name;
```



where, **n** is the number of bits to allocate, and **name** is the name assigned.

Initializing

By default, when a bitset of a given size is created, all of its bits are unset i.e. 0. We can initialize it to **some value** simply by passing it to its constructor.

```
#include <bits/stdc++.h>
using namespace std;

int main() {

    // Default Initialization
    bitset<5> bnum(18);
    cout << bnum;
    return 0;
}
```



Here, the bitset **bnum** is of size 5 bits and stores the decimal value 18 in binary form.

Binary numbers are also represented as strings, so bitset also provide facility to initialize itself from the **strings**.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
```



```
// Initialize bitset with value
bitset<5> bs2("10010");

cout << bs2 << endl;
return 0;
}
```

The string should represent a valid binary number i.e. **all characters should be 0 or 1**. Otherwise, an error may occur.

Accessing Individual Bits

We can access individual bits in a bitset using the `[]` operator, just like an array.

Example:

```
#include <bits/stdc++.h>
using namespace std;

int main() {

    // 18 = (10010)
    bitset<5> bs(18);

    // Check 3rd bit
    cout << bs[2] << endl;

    // Check 5th bit
    cout << bs.test(4);
    return 0;
}
```

Output

```
0
1
```

Setting, Resetting and Flipping

Setting means making the bit at particular position 1 and resetting means making it 0. These operations can be done using [`set\(\)`](#) and [`reset\(\)`](#) function. The [`flip\(\)`](#).

function can be used to set the bit if it's not set and unset if it is set.

Example:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    bitset<5> bs(18);

    // Set 1st bit
    bs.set(0);
    cout << bs << endl;

    // Reset 2nd bit
    bs.reset(1);
    cout << bs << endl;

    // Flip 5th bit
    bs.flip(4);
    cout << bs;
    return 0;
}
```

Output

```
10011
10001
00001
```

Bitset Operators

Bitset objects can work with all the bitwise operators to provide seamless replacement integration in the code.

Operator	Operation
&	Bitwise AND

Operator	Operation
	Bitwise OR
^	Bitwise XOR
>>=	Binary Right shift and assign
<<=	Binary Left shift and assign
&=	Assign the value of bitwise AND to the first bitset.
=	Assign the value of bitwise OR to the first bitset.
^=	Assign the value of bitwise XOR to the first bitset.
~	Bitwise NOT

Example:

```
#include <bits/stdc++.h>
using namespace std;

int main() {

    // 18 = (10010)
    bitset<5> bs1(18);

    // 5 = (00101)
    bitset<5> bs2(5);

    // AND Operator
    cout << (bs1 & bs2) << endl;

    // OR Operator
    cout << (bs1 | bs2) << endl;
```

```
// XOR operator
cout << (bs1 ^ bs2);
return 0;
}
```

Output

```
00000
10111
10111
```

Why Bitset is Preferred

- Memory Efficient- Stores bits compactly using less space than arrays of bools.
- Fast Operations- Bitwise operations like AND, OR, XOR are very fast.
- Easy to Use- Provides built-in functions like count(), any(), none(), and indexing with[].

Difference between std::bitset and std::vector<bool> and an array of bool

Vector of bool and array of bool can also be implemented to store a sequence of boolean values like bitset but there are some differences between each implementation:

Parameter	bitset	vector of bool	array of bool
Definition	A class template consisting of a sequence of bits stored such that each bit occupies 1 bit of memory.	A variation of vectors of C++ STL in which each element is of size 1 bit and is of type bool	A fixed size contiguous collection of bool data elements.
Size	Fixed Size.	Dynamic Size.	Fixed Size.

Parameter	bitset	vector of bool	array of bool
Memory	A single element occupies 1 bit of memory.	A single element occupies 1 bit of memory.	A single element occupies 1 byte of memory.
Speed	Same	Same	Faster

[Comment](#)
[More info](#)
[Advertise with us](#)

Explore

C++ Programming Language

🕒 5 min read

C++ Overview



C++ Basics



C++ Variables and Constants



C++ Data Types and Literals



C++ Operators



C++ Input/Output



C++ Control Statements



C++ Functions



C++ Pointers and References



Do Not Sell or Share My Personal Information